

Thoughts on a conservative terse syntax for constraints

ISO/IEC JTC1 SC22 WG21 P1142R0

To: EWG (informative only)

Compatible. Principled. Useful.

This document is *not* a proposal for a change. It merely serves as a record of ideas that the author has considered while many of us were developing a terse concepts syntax ahead of the San Diego meeting. The author endorses the resulting joint proposal, *Yet another approach for constrained declarations* (YAACD), which should be our first and foremost concentrated effort. Where YAACD strives for a minimal extension that avoids invention as much as possible, this paper describes a larger design that retains more of the features from the Concepts TS. If we come to revisit that design space in the future, it might be nice to be able to refer to these past thoughts.

Contents

1. [Summary](#)
2. [In a nutshell, by example](#)
3. [Scope, features, limitations](#)
4. [Definitions](#)
5. [Terse syntaxes](#)
6. [Rationale](#)
7. [Alternatives](#)
8. [Final words](#)
9. [References](#)

Summary

This paper describes a compromise design for a terse concept syntax, informed by the discussions around several past attempts. As a result, this design...

- is *useful* for many, but not all, common uses of concepts;
- is *regular and predictable*, largely free of ambiguities (completely so if a certain style is followed); and
- *compatible*, with only a minor breaking change with respect to the post-Toronto working paper and only in uncommon cases.

The central design decisions are a response to previous discussions, notably those following [P0745R1](#) and [P1079R0](#). They are:

- **No introducer syntax.** Introducers are an intriguing method to write certain constraint relationships among types, but the committee has consistently been ambivalent about any particular proposed syntax. Each suggestion allows for appealing code in certain situations, but suffers either from ambiguities or from a proliferation of novel punctuators in other situations. This proposal simply does not provide introducers; complex constraints need to be written with a *requires-clause*.
- **Terse function parameters use `auto`.** After several rounds of discussion around terse function template syntax, the committee seems to have settled on wanting a syntactic marker to distinguish templates from non-templates, and moreover to tell which function parameters are templated. The solution to consistently mark each templated parameter with `auto` appears to be acceptable to most people: `void sort(Sortable auto& container);`
- **Conservative common case.** For unary type concepts, the syntax from the working paper is retained: `template<Sortable T>`
- **Unambiguous general case.** When `C` is a concept, we never want `template<C T>` to make `T` anything other than a type. To effect this, unary concepts of non-type or template kind require a mandatory noun (in the terminology of P0807). The noun is `template` for template concepts and either `auto` or a type name for non-type concepts. Optionally, type concepts may (but do not have to) use the noun `typename` or `class`; doing so makes the syntax entirely unambiguous (even when `C` above is not a concept).

Breaking changes: Only the last point constitutes a breaking change with respect to the post-Toronto working paper and the Concepts TS. In those, `C` is allowed to be a non-type or template concept, making `T` respectively a non-type or template

template-parameter. In the design of this paper, a mandatory noun must be inserted. Non-type and template *template-parameters* are comparatively rare, so we consider this breakage to be minor.

In a nutshell, by example

Short syntax	Equivalent <i>requires-clause</i>
<pre>auto gcd(Integer auto a, Integer auto b);</pre>	<pre>template<typename T1, typename T2> requires Integer<T1> && Integer<T2> auto gcd(T1 a, T2 b);</pre>
<pre>template<Iterator I> void sort(I first, I last);</pre>	<pre>template<typename I> requires Iterator<I> void sort(I first, I last);</pre>
<pre>template<Even int N> void AddConsecutivePairs(const auto (&arr) [N], OutputIterator auto it);</pre>	<pre>template<int N, typename T, typename Out> requires Even<N> && OutputIterator<Out> void AddConsecutivePairs(const T (&arr)[N], Out it);</pre>
<pre>template<ContainerLike template Tmpl, typename T> struct MyAdaptor;</pre>	<pre>template<template<typename> typename Tmpl, typename T> requires ContainerLike<Tmpl> struct MyAdaptor;</pre>
Constrained variable and return type declarations	
<i>[Note: Only type concepts can be applied to constrained declarations (variables, function return types, function parameters). – end note]</i>	
<pre>Sortable auto x = f();</pre>	Type of <code>x</code> is deduced, ill-formed if it does not satisfy the type concept. Roughly equivalent to <code>static_assert(Sortable<decltype(x)>)</code> .
<pre>Sortable auto f() { /* ... */ }</pre>	Return type of <code>f</code> is deduced from the body, ill-formed if it does not satisfy the type concept.

Scope, features, limitations

This paper describes a set of “terse concept syntaxes” based on the *adjective syntax* that has been developed in [P0807](#). It consists of the core proposal of P0807, plus the future extensions for constrained function parameters, function return types and variable declarations discussed therein, and the modification to make `typename/class` optional.

They key design points of the resulting proposal are:

- Unary concepts only (defined below). Retains the *partial-concept-ids* from the working paper (no breaking change there).
- No ambiguities: the “kind” of a template parameter is always syntactically marked (via type name, `auto`, `typename/class` or `template`) (unless `typename/class` is made optional).
- Unused names can be omitted with no change in meaning (just like everywhere else).
- Names are introduced by the familiar template head. No new introducer syntax.
- Binary concepts require the full *requires-clause* syntax: `template<typename I, typename S> requires Iterator<I, S>`.

The upside of the proposed approach is that it is a minimal extension that retains many of the familiar properties of template syntax. The major downside is that it can only handle unary concepts. There is no short syntax for complex concepts whatsoever. Whether this is a problem depends on how important we consider unary concepts relative to all uses of concepts, i.e. whether we can help a large fraction of the use cases.

Details appertaining to *partial-concept-ids* are not spelled out in this paper. P0807 contains full examples.

Details around constraining `auto` declarations that are part of structured bindings, trailing return types or `decltype(auto)` are not discussed here, but should follow the conclusions of YAACD.

Definitions

A concept is a *unary concept in the context where it is used* if its prototype parameter (or parameter pack) is the only free parameter (or pack), and all further parameters (if any) have been bound to a fixed set of arguments (via a *partial-concept-id*). The kind of the prototype parameter (pack) determines the kind of the unary concept (names following P0807):

Kind of concept	Example
Type concept	<pre>template<typename T> concept Integral = /* ... */;</pre>
Value concept	<pre>template<int N> concept Even = N % 2 == 0;</pre>
Auto concept	<pre>template<auto N> concept EvenIntegral = Integral<decltype(N)> && N % 2 == 0;</pre>
Template concept	<pre>template<template<typename> typename Tmpl> concept Foo = /* ... */;</pre>

Four kinds of concept

Concepts that are not unary in a particular context are called *complex concepts*. A constraint that uses a complex concept or that uses multiple concepts (or both) is called a *complex constraint*. [Example: `template<typename X, typename Y, typename Z> requires C1<X, Y> && C2<Z> && C3<Z>` is a complex constraint. — *end example*]

Terse syntaxes

Constrained parameters

This is the content of P0807. Adjective syntax can be applied to a *constrained-parameter* of all kinds. Note in particular that value, type and auto concepts can be applied to value, type and auto parameters in various combinations.

```
template <Integral      typename T> struct X;  // type concept, type parameter
template <Integral      class T> struct X;    // synonym for the above
template <Integral      T> struct X;          // same as above, "typename"/"class" is optional
template <Even          int    N> struct X;    // value concept, value parameter
template <EvenIntegral  auto   N> struct X;    // auto concept, auto parameter
template <EvenIntegral  int    N> struct X;    // auto concept, value parameter, deduce "auto" as "int"
template <Even          auto   N> struct X;    // value concept, auto parameter, ill-formed unless decltype(N)
template <Integral      auto   N> struct X;    // type concept, auto parameters, means "requires Integral<decl
```

Note that constraints can be relaxed by dropping the concept name (except that an omitted “`typename`” needs to be reintroduced).

Complex concepts cannot be used with this syntax and instead need the full *requires-clause* syntax.

Terse function templates

The goal of a terse function template syntax is to use the function parameters to turn the function into a function template. The only part of the template that can be parameterised in this context is the function parameter *type*, and thus we only need to consider type concepts here.

The proposed terse syntax does not allow the deduced types to be named. If a named template parameter is required, the traditional template head should be used instead.

```

void f(MyInt n); // not a template because no "auto" or "template"
void f(Integral auto n); // template because "auto"
void f(Integral auto n1, Integral auto n2); // two separately deduced types
template<Integral typename T> void f(T n1, T n2); // one type
void f(MyInt n1, Integral auto n2); // function template, n1 is converted, n2 is deduced
void f(auto n1, MyInt n2, auto n3); // function template, two unconstrained template parameters

template<typename I, typename S> requires Iterator<I, S>
    void f(I iter, S sentinel); // template constrained by complex concept (no short syntax available)

// mix and match (constrained parameters and terse function syntax)
dispatch_sort(v, [<Integral typename T>(T begin, T end, const Predicate auto& p) { /* ... */ });

```

Constrained declarations

This part of the syntax allows constrained forms of deduced types for variable and function return type declarations.

```

Integral auto f(int n, double x) { return x * n; }

auto x = f(); // unconstrained
Integral auto y = g(); // constrained type

```

Rationale

How did we get here? The short syntax forms that appear in the Concepts TS are appealingly short, but suffer from ambiguities: constrained template parameters do not syntactically show whether they are of type, non-type or template kind, and function parameters do not syntactically signal whether the function is in fact a function template. When the bulk of the concepts extensions were adopted in the IS working paper in Toronto 2017, the short syntax forms other than constrained parameters were excluded, as the committee did not feel confident about the syntax.

A number of proposals for short syntax forms have been made since Toronto. Many of them involved new syntax to *introduce* names for the types that were being constrained, and in the process addressing the aforementioned ambiguities to varying degrees. However, two main questions kept coming back:

- How can function templates be distinguished from non-templates?
- How verbose is the common case?

None of the proposals were entirely satisfying:

- `void sort(Shape s, Number n);` — the syntax from the Concepts TS, which makes *no* distinction between templates and non-templates (but in which the common case is pleasantly short).
- `template void sort(Shape s, Number n);` — from P1079, this syntax is still short, but it leaves functions with more than one parameter ambiguous as to *which* parameters are templated.
- `void sort(Shape s, Number{T} n);` — from P0745, this syntax is unambiguous and fairly short, and offers additional expressiveness by allowing the deduced type to be named. However, the syntax is somewhat novel, and the full proposal puts empty braces (`{}`) in several common places. The proposal also contains situations where leaving out a parameter name changes the meaning of a declaration.

At the same time, proposals had been made for an “adjective” syntax, which adds the concept name as an additional, whitespace separated token in the declaration. This syntax is often more verbose, but it can be made very regular without ambiguities. While previous proposals did not gain traction, one particular adjective-based approach is being considered now to resolve the above problems:

- `void sort(Shape s, Number auto n);` — adjective syntax, the noun `auto` signals that this is a template.

Once we allow a value to have its (deduced) type constrained by a concept adjective, it is only a short jump to allow the same for deduced non-type template parameters: `template<Number auto N>`. But in this position, `Number` need not just be a type-concept, but it can also reasonably be an `auto`-concept (i.e. a concept that constrains the value): `template<EvenIntegral auto N>`. Once we have value-concept adjectives, we should allow them not just on `auto`, but on concrete types, too: `template<Even int N>`.

This gives us an inroad for an adjective syntax for constrained parameters. When non-type parameters take their constraint as adjectives, we are closer to avoiding ambiguities. We can take this further and move the constraints into adjective position for the other parameter kinds. For template *template-parameters*, this means saying `template<ContainerLike template Tmpl>`. (We can even allow the full template head here to make `Tmpl` a specific template.) This only leaves type parameters. For consistency, they should also support an adjective style: `template<Iterator class T>` (or `typename`). However, since this kind of parameter is by far the most widely used kind, the `typename/class` noun is optional.

When all is said and done, we end up with mandatory adjective syntax for constrained function parameters, function return types and variable declarations as a new feature (where the noun is always `auto` and the constraint is always via type-concept). We also make the noun mandatory for non-type and template *template-parameters*, which is a breaking change, but those kinds of parameters are relatively rare. The result is a coherent syntax that works predictably and is largely free from ambiguities. And it can be made fully unambiguous if the `typename/class` noun is used.

Alternatives

- Compared to the post-Toronto working paper, we could only add terse declaration syntax (constrained `auto`), and not change anything else (i.e. leave constrained parameters as they were). This means that we have a function parameter or variable declaration `Number auto n` of constrained type, but we would not be able to constrain a non-type template parameter. The syntax `template<Number N>` makes `N` a type, not a value, and a value `template<auto N>` could only be constrained by a *requires-clause*. By contrast with the design here presented, this alternative approach does not allow simply dropping the concept to get to a less constrained (but still valid) parameter.
- Add the terse function syntax, and disallow constrained parameters for anything other than type-concepts. This avoids ambiguities among the constrained parameters, but it feels needlessly restrictive. It is a breaking change from the working paper without a good replacement. If `template<auto N>` is valid, and declarations of the form `auto N` can be constrained elsewhere, it would be surprising that the template parameter cannot be constrained in the same way.

Comparison with YAACD: The main difference between the present design and that proposed for San Diego is that the present design allows the use of constrained parameters for template *template-parameters* (using the mandatory noun `template`), it allows non-type parameters to have non-type constraints (auto- and value-concepts), and it allows optionally to specify the `typename/class` noun.

Final words

- Terse syntax is situational. If you need to introduce a deduced name (e.g. to use the name repeatedly), the traditional template head syntax is preferable. Constrained declarations have no introducer alternative; `decltype` must be used to obtain the deduced type.
- “Saying `Integral typename T` is verbose.” Yes, it is, but keeping the noun mandatory makes the syntax regular and predictable. When `typename/class` is omitted, we cannot tell whether `template<A B>` is a type parameter constrained by concept `A` or a non-type parameter of type `A`. (But we can live with this in many cases.) This proposal leaves the choice to the user.
- Is this approach viable? It depends on how much more common unary concepts are than complex concepts. Complex concepts may benefit from an introducer syntax, which this approach will not achieve. If complex concepts are comparatively rare, then the adjective approach helps a large number of use cases without introducing very novel syntax.

References

- [N4549](#): Andrew Sutton, *Programming Languages — C++ Extensions for Concepts*, 2015-07-27
- [P0745R1](#): Herb Sutter, *Concepts in-place syntax*, 2018-04-29
- [P0807R0](#): Thomas Köppe, *An Adjective Syntax for Concepts*, 2017-10-12
- [P1079R0](#): Bjarne Stroustrup, *A minimal solution to the concepts syntax problem*, 2018-05-06
- [YAACD]: Ville Voutilainen et al., *Yet another approach for constrained declarations*, 2018-06-25